Solution - Data Engineering

CAPSTONE PROJECT


Global E-Commerce Analytics Platform

🚀 STEP 1 – EXTRACT DATA FROM THE DUMMYJSON API

🎯 Objective

Download data from every required API endpoint and save each response as a JSON file.

At this stage:

- ✅ Connect to the API
- ✅ Read JSON data
- ✅ Handle pagination (if needed)
- ✅ Save the JSON exactly as received
- ❌ Do NOT clean the data yet
- ❌ Do NOT load into Snowflake yet

This is the **data ingestion layer**.

Step 1.1 Create the Project Folder

Create the following project structure.

```
Capstone_Project/
│
├── extract/
│
├── data/
│   └── raw/
│
├── requirements.txt
│
└── README.md
```

Step 1.2 Create a Virtual Environment

Open the terminal inside the project folder.

Windows

```
python -m venv .venv
```

Activate it:

```
.venv\Scripts\activate
```

Step 1.3 Install Required Packages

Install the libraries you'll need.

```
pip install requests python-dotenv
```

Freeze the dependencies:

```
pip freeze > requirements.txt
```

Your requirements.txt should look similar to:

```
requests
python-dotenv
```

Step 1.4 Create the Raw Data Folder

Create:

```
data/
└── raw/
```

Later this folder will contain:

```
raw/
users.json
products.json
carts.json
posts.json
comments.json
quotes.json
recipes.json
categories.json
```

Step 1.5 Create the Extraction Script

Inside the extract folder create:

```
extract/
    extract_api.py
```

Instead of creating many small scripts, we'll build **one reusable extractor** that downloads every endpoint automatically. This is closer to professional practice.

Step 1.6 Add the Python Code

Create extract/extract_api.py:

```python
import json
import os
from pathlib import Path

import requests

# ----------------------------------------
# Configuration
# ----------------------------------------

BASE_URL = "https://dummyjson.com"

ENDPOINTS = {
    "users": "users",
    "products": "products",
    "carts": "carts",
    "posts": "posts",
    "comments": "comments",
    "quotes": "quotes",
    "recipes": "recipes",
    "categories": "products/categories"
}

RAW_FOLDER = Path("data/raw")
RAW_FOLDER.mkdir(parents=True, exist_ok=True)


# ----------------------------------------
# Download Function
# ----------------------------------------
```

```
def download_endpoint(name, endpoint):
    url = f"{BASE_URL}/{endpoint}"

    print(f"Downloading {name}...")

    response = requests.get(url)

    response.raise_for_status()

    data = response.json()

    output_file = RAW_FOLDER / f"{name}.json"

    with open(output_file, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=4)

    print(f"Saved -> {output_file}")


# ----------------------------------------
# Main
# ----------------------------------------

def main():

    for name, endpoint in ENDPOINTS.items():
        download_endpoint(name, endpoint)

    print("\nFinished downloading all datasets.")


if __name__ == "__main__":
    main()
```

Step 1.7 Run the Script

From the project root:

```
python extract/extract_api.py
```

Step 1.8 Expected Output

Terminal:

```
Downloading users...
Saved -> data/raw/users.json

Downloading products...
Saved -> data/raw/products.json

Downloading carts...
Saved -> data/raw/carts.json

Downloading posts...
Saved -> data/raw/posts.json

Downloading comments...
Saved -> data/raw/comments.json

Downloading quotes...
Saved -> data/raw/quotes.json

Downloading recipes...
Saved -> data/raw/recipes.json

Downloading categories...
Saved -> data/raw/categories.json

Finished downloading all datasets.
```

Step 1.9 Verify the Output

Your project should now look like:

```
Capstone_Project/
│
├── data/
│   └── raw/
│       ├── users.json
│       ├── products.json
│       ├── carts.json
│       ├── posts.json
│       ├── comments.json
│       ├── quotes.json
│       ├── recipes.json
│       └── categories.json
│
├── extract/
│   └── extract_api.py
│
```

```
├── requirements.txt
│
└── README.md
```

📚 What You Learned

By completing Step 1, you've learned how to:

- Use the requests library to call a REST API.
- Read JSON responses from HTTP endpoints.
- Save API responses as formatted JSON files.
- Organize a data engineering project with a reusable extraction script.
- Prepare raw data for the next stages of the pipeline.

🚀 STEP 2 — STORE RAW DATA (DATA LAKE LAYER)

🎯 Objective

Store every API response **exactly as received**.

Do not:

- ❌ Remove columns
- ❌ Rename fields
- ❌ Convert data types
- ❌ Filter records

This layer is your **Raw Data Lake**, preserving the original API responses.

Step 2.1 Update the Folder Structure

Instead of placing all files directly under data/raw, create a folder for each endpoint.

```
Capstone_Project/
│
```

```
├── data/
│   └── raw/
│       ├── users/
│       ├── products/
│       ├── carts/
│       ├── posts/
│       ├── comments/
│       ├── quotes/
│       ├── recipes/
│       └── categories/
```

Step 2.2 Timestamp the Files

Each execution should create a new file.

Example:

```
users_2026_07_06_101530.json
products_2026_07_06_101530.json
carts_2026_07_06_101530.json
```

This allows you to:

- Track historical data
- Reprocess old data if needed
- Audit what was received from the API

Step 2.3 Replace extract_api.py

Replace your previous script with the following version.

```python
import json
from datetime import datetime
from pathlib import Path

import requests

BASE_URL = "https://dummyjson.com"

ENDPOINTS = {
    "users": "users",
    "products": "products",
    "carts": "carts",
```

```
    "posts": "posts",
    "comments": "comments",
    "quotes": "quotes",
    "recipes": "recipes",
    "categories": "products/categories"
}

RAW_ROOT = Path("data/raw")


def download_endpoint(name, endpoint):
    url = f"{BASE_URL}/{endpoint}"

    print(f"Downloading {name}...")

    response = requests.get(url)
    response.raise_for_status()

    data = response.json()

    endpoint_folder = RAW_ROOT / name
    endpoint_folder.mkdir(parents=True, exist_ok=True)

    timestamp = datetime.now().strftime("%Y_%m_%d_%H%M%S")

    filename = f"{name}_{timestamp}.json"

    filepath = endpoint_folder / filename

    with open(filepath, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=4)

    print(f"Saved -> {filepath}")


def main():
    for name, endpoint in ENDPOINTS.items():
        download_endpoint(name, endpoint)

    print("\nAll datasets successfully saved.")


if __name__ == "__main__":
    main()
```

Step 2.4 Run the Script

```
python extract/extract_api.py
```

Step 2.5 Expected Output

```
Downloading users...
Saved -> data/raw/users/users_2026_07_06_183001.json

Downloading products...
Saved -> data/raw/products/products_2026_07_06_183002.json

Downloading carts...
Saved -> data/raw/carts/carts_2026_07_06_183002.json

...

All datasets successfully saved.
```

Step 2.6 Verify the Folder Structure

After running the script, your project should look similar to:

```
Capstone_Project/
│
├── data/
│   └── raw/
│       ├── users/
│       │   ├── users_2026_07_06_183001.json
│       │   ├── users_2026_07_07_090015.json
│       │
│       ├── products/
│       │   ├── products_2026_07_06_183002.json
│       │
│       ├── carts/
│       ├── posts/
│       ├── comments/
│       ├── quotes/
│       ├── recipes/
│       └── categories/
│
├── extract/
│   └── extract_api.py
```

Notice that each execution creates a **new timestamped file** instead of replacing the previous one.

Step 2.7 Why This Is a Best Practice

This design offers several advantages:

Benefit	Explanation
Historical snapshots	Keep every API extract for auditing and reprocessing.
Data recovery	Restore data from a previous run if needed.
Data lineage	Trace exactly when each dataset was collected.
Industry standard	This is how raw data is commonly stored in production data lakes.

📚 What You Learned

By completing Step 2, you've learned how to:

- Build a raw data lake structure.
- Preserve original API responses without modification.
- Create timestamped files for historical tracking.
- Organize data by source for easier maintenance.

🚀 STEP 3 — LOAD RAW DATA INTO SNOWFLAKE

🎯 Objective

Create the Snowflake environment and load all raw JSON files into the **RAW** schema.

Step 3.1 Project Structure

Add a new folder:

```
Capstone_Project/
│
```

```
├── snowflake/
│   ├── create_database.sql
│   ├── create_raw_tables.sql
│   ├── load_raw_data.py
│   └── config.py
```

Step 3.2 Install Required Libraries

```
pip install snowflake-connector-python python-dotenv
```

Update:

```
pip freeze > requirements.txt
```

Step 3.3 Create a .env File

Create .env in the project root.

```
SNOWFLAKE_ACCOUNT=YOUR_ACCOUNT
SNOWFLAKE_USER=YOUR_USERNAME
SNOWFLAKE_PASSWORD=YOUR_PASSWORD
SNOWFLAKE_ROLE=ACCOUNTADMIN
SNOWFLAKE_WAREHOUSE=COMPUTE_WH
SNOWFLAKE_DATABASE=CAPSTONE_DB
SNOWFLAKE_SCHEMA=RAW
```

Step 3.4 Create the Database

Create snowflake/create_database.sql

```
CREATE DATABASE IF NOT EXISTS CAPSTONE_DB;

USE DATABASE CAPSTONE_DB;

CREATE SCHEMA IF NOT EXISTS RAW;
CREATE SCHEMA IF NOT EXISTS STAGING;
CREATE SCHEMA IF NOT EXISTS MART;
```

Run this script in your Snowflake worksheet.

Step 3.5 Create the Raw Tables

Switch to the RAW schema:

```
USE DATABASE CAPSTONE_DB;
USE SCHEMA RAW;
```

Create one table for each endpoint.

```
CREATE OR REPLACE TABLE RAW_USERS (
    RAW_DATA VARIANT
);

CREATE OR REPLACE TABLE RAW_PRODUCTS (
    RAW_DATA VARIANT
);

CREATE OR REPLACE TABLE RAW_CARTS (
    RAW_DATA VARIANT
);

CREATE OR REPLACE TABLE RAW_POSTS (
    RAW_DATA VARIANT
);

CREATE OR REPLACE TABLE RAW_COMMENTS (
    RAW_DATA VARIANT
);

CREATE OR REPLACE TABLE RAW_QUOTES (
    RAW_DATA VARIANT
);

CREATE OR REPLACE TABLE RAW_RECIPES (
    RAW_DATA VARIANT
);

CREATE OR REPLACE TABLE RAW_CATEGORIES (
    RAW_DATA VARIANT
);
```

Why use VARIANT?

Snowflake's VARIANT type stores semi-structured data such as JSON without requiring a predefined schema. This is the ideal choice for the raw ingestion layer.

Step 3.6 Create the Connection Configuration

Create snowflake/config.py

```
import os
from dotenv import load_dotenv

load_dotenv()

SNOWFLAKE_CONFIG = {
    "account": os.getenv("SNOWFLAKE_ACCOUNT"),
    "user": os.getenv("SNOWFLAKE_USER"),
    "password": os.getenv("SNOWFLAKE_PASSWORD"),
    "role": os.getenv("SNOWFLAKE_ROLE"),
    "warehouse": os.getenv("SNOWFLAKE_WAREHOUSE"),
    "database": os.getenv("SNOWFLAKE_DATABASE"),
    "schema": os.getenv("SNOWFLAKE_SCHEMA"),
}
```

Step 3.7 Create the Loader Script

Create snowflake/load_raw_data.py

```
import json
from pathlib import Path

import snowflake.connector

from config import SNOWFLAKE_CONFIG

RAW_PATH = Path("../data/raw")

TABLES = {
    "users": "RAW_USERS",
    "products": "RAW_PRODUCTS",
    "carts": "RAW_CARTS",
    "posts": "RAW_POSTS",
    "comments": "RAW_COMMENTS",
    "quotes": "RAW_QUOTES",
```

```
    "recipes": "RAW_RECIPES",
    "categories": "RAW_CATEGORIES",
}

conn = snowflake.connector.connect(**SNOWFLAKE_CONFIG)

cursor = conn.cursor()

for folder, table in TABLES.items():

    files = sorted((RAW_PATH / folder).glob("*.json"))

    if not files:
        continue

    latest = files[-1]

    with open(latest, "r", encoding="utf-8") as f:
        data = json.load(f)

    cursor.execute(f"DELETE FROM {table}")

    cursor.execute(
        f"INSERT INTO {table}(RAW_DATA) SELECT PARSE_JSON(%s)",
        (json.dumps(data),),
    )

    print(f"Loaded {latest.name} -> {table}")

cursor.close()
conn.close()
```

Step 3.8 Execute the Loader

From the project root:

```
python snowflake/load_raw_data.py
```

Expected output:

```
Loaded users_2026_07_06_183001.json -> RAW_USERS
Loaded products_2026_07_06_183002.json -> RAW_PRODUCTS
Loaded carts_2026_07_06_183002.json -> RAW_CARTS
...
```

Step 3.9 Verify the Data

Run a simple query:

```
SELECT *
FROM RAW_USERS;
```

You should see a single VARIANT column containing the original JSON document.

To inspect fields:

```
SELECT
    RAW_DATA:users[0].firstName AS FIRST_NAME,
    RAW_DATA:users[0].lastName AS LAST_NAME,
    RAW_DATA:users[0].email AS EMAIL
FROM RAW_USERS;
```

This confirms the JSON has been loaded successfully.

📚 What You Learned

By completing Step 3, you have:

- Created a Snowflake database and schemas.
- Stored raw API responses in VARIANT columns.
- Connected Python to Snowflake using environment variables.
- Loaded JSON files into raw tables without modifying the data.

🚀 STEP 4 — TRANSFORM DATA WITH DBT

🎯 Objective

Convert raw JSON into clean relational tables.

We'll build the following dbt architecture:

```
RAW (Snowflake)
       |
       ▼
Sources
```

```
       |
       ▼
Staging
       |
       ▼
Intermediate
       |
       ▼
Mart
```

Configure macros/generate_schema_name.sql

```
{% macro generate_schema_name(custom_schema_name, node) %}

    {% if custom_schema_name is none %}
        {{ target.schema }}
    {% else %}
        {{ custom_schema_name | trim }}
    {% endif %}

{% endmacro %}
```

Step 4.1 Create the dbt Project

From the project root:

```
dbt init ecommerce_dbt
```

Choose:

```
1) snowflake
```

Your project structure becomes:

```
Capstone_Project/
│
├── ecommerce_dbt/
│    ├── models/
│    ├── macros/
│    ├── tests/
│    ├── snapshots/
│    ├── seeds/
│    ├── analyses/
```

```
│   ├── dbt_project.yml
│
├── snowflake/
├── extract/
├── data/
```

Step 4.2 Configure profiles.yml

Create (or update):

```
~/.dbt/profiles.yml
```

```
ecommerce_dbt:

  target: dev

  outputs:

    dev:
      type: snowflake

      account: "{{ env_var('SNOWFLAKE_ACCOUNT') }}"
      user: "{{ env_var('SNOWFLAKE_USER') }}"
      password: "{{ env_var('SNOWFLAKE_PASSWORD') }}"

      role: "{{ env_var('SNOWFLAKE_ROLE') }}"
      warehouse: "{{ env_var('SNOWFLAKE_WAREHOUSE') }}"
      database: "{{ env_var('SNOWFLAKE_DATABASE') }}"
      schema: STAGING

      threads: 4
```

Since you already configured your .env, dbt will automatically use those variables.

Step 4.3 Test the Connection

```
dbt debug --profiles-dir .
```

Expected:

```
Connection test: OK
```

Step 4.4 Organize the Models

Inside models/ create:

```
models/
├── sources/
├── staging/
├── intermediate/
├── marts/
└── schema.yml
```

Step 4.5 Create dbt Sources

Create:

```
models/sources/sources.yml
```

```yaml
version: 2

sources:

  - name: raw

    database: CAPSTONE_DB

    schema: RAW

    tables:

      - name: raw_users
      - name: raw_products
      - name: raw_carts
      - name: raw_posts
      - name: raw_comments
      - name: raw_quotes
      - name: raw_recipes
      - name: raw_categories
```

Now dbt knows where the raw tables are.

Step 4.6 Build the First Staging Model

Create:

`models/staging/stg_users.sql`

```
WITH source AS (

    SELECT RAW_DATA
    FROM {{ source('raw','raw_users') }}
),

flatten_users AS (

    SELECT
        VALUE AS user_data

    FROM source,
    LATERAL FLATTEN(input => RAW_DATA:users)
)

SELECT

    user_data:id::INTEGER               AS user_id,
    user_data:firstName::STRING         AS first_name,
    user_data:lastName::STRING          AS last_name,
    user_data:maidenName::STRING        AS maiden_name,
    user_data:age::INTEGER              AS age,
    user_data:gender::STRING            AS gender,
    user_data:email::STRING             AS email,
    user_data:phone::STRING             AS phone,
    user_data:username::STRING          AS username,
    user_data:bloodGroup::STRING        AS blood_group,
    user_data:eyeColor::STRING          AS eye_color,
    user_data:birthDate::DATE           AS birth_date,
    user_data:height::FLOAT             AS height,
    user_data:weight::FLOAT             AS weight,
    user_data:image::STRING             AS image_url
FROM flatten_users
```

This model:

- Reads the JSON document.

- Flattens the users array.
- Extracts individual attributes.
- Casts each field to the proper data type.

Step 4.7 Create a Product Staging Model

Create:

`models/staging/stg_products.sql`

```
WITH source AS (

    SELECT RAW_DATA

    FROM {{ source('raw','raw_products') }}

),

flatten_products AS (

    SELECT VALUE AS product

    FROM source,
    LATERAL FLATTEN(input => RAW_DATA:products)

)

SELECT

    product:id::INTEGER         AS product_id,

    product:title::STRING       AS product_name,

    product:category::STRING    AS category,

    product:brand::STRING       AS brand,

    product:price::NUMBER       AS price,

    product:rating::FLOAT       AS rating,

    product:stock::INTEGER      AS stock

FROM flatten_products
```

Step 4.8 Create schema.yml

```
version: 2

models:

  - name: stg_users

    columns:

      - name: user_id
        tests:
          - unique
          - not_null

      - name: email
        tests:
          - unique
          - not_null

  - name: stg_products

    columns:

      - name: product_id
        tests:
          - unique
          - not_null

      - name: product_name
        tests:
          - not_null
```

These tests automatically validate your data.

Step 4.9 Run dbt

```
dbt run --profiles-dir .
```

Expected output:

```
Running with dbt...
```

```
Found 2 models

Created model STAGING.STG_USERS

Created model STAGING.STG_PRODUCTS

Completed successfully
```

Step 4.10 Execute Tests

```
dbt test --profiles-dir .
```

Example:

```
PASS not_null_stg_users_user_id

PASS unique_stg_users_email

PASS unique_stg_products_product_id

Completed successfully
```

Step 4.11 Generate Documentation

```
dbt docs generate --profiles-dir .
```

Serve the documentation:

```
dbt docs serve --profiles-dir .
```

You'll get an interactive website showing:

- Sources
- Models
- Column descriptions
- Data lineage
- Tests

- Dependencies

📚 What You Learned

By completing Step 4, you have:

- Connected dbt to Snowflake.
- Declared raw sources.
- Flattened nested JSON arrays using LATERAL FLATTEN.
- Built clean staging models with typed columns.
- Added automated data quality tests.
- Generated interactive documentation for your transformation layer.

🚀 STEP 5 — BUILD THE STAR SCHEMA

🎯 Objective

Create a dimensional model optimized for reporting and Power BI.

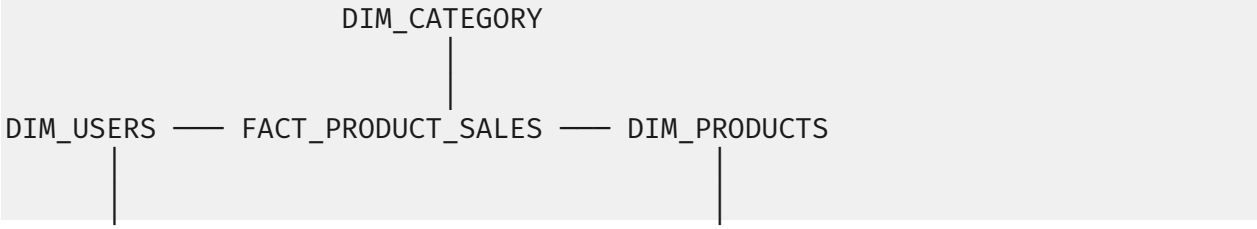
Instead of querying raw JSON directly, analysts will query **fact** and **dimension** tables.

What is a Star Schema?

A Star Schema consists of:

- **Fact Tables** → Store measurable business events (sales, orders, carts).
- **Dimension Tables** → Store descriptive information (users, products, categories).

Our schema will look like this:



```
      └──────── FACT_CARTS ──────────────┘
                        │
                  DIM_COMPANY
                        │
                  DIM_ADDRESS
```

⭐ Fact tables:

- FACT_CARTS
- FACT_PRODUCT_SALES

📦 Dimension tables:

- DIM_USERS
- DIM_PRODUCTS
- DIM_CATEGORY
- DIM_COMPANY
- DIM_ADDRESS

```
models/
│
├── staging/
│
├── intermediate/
│
├── marts/
│   ├── dimensions/
│   └── facts/
```

Step 5.2 Create DIM_USERS

Create:

```
models/marts/dimensions/dim_users.sql
```

```
SELECT

    user_id,
    first_name,
```

```
    last_name,
    age,
    gender,
    email,
    phone,
    username,
    birth_date,
    height,
    weight,
    blood_group,
    eye_color,
    image_url
FROM {{ ref('stg_users') }}
```

Run:

```
dbt run --select dim_users --profiles-dir .
```

Step 5.3 Create DIM_PRODUCTS

Create:

```
models/marts/dimensions/dim_products.sql
```

```
SELECT

    product_id,
    product_name,
    brand,
    category,
    price,
    rating,
    stock
FROM {{ ref('stg_products') }}
```

Step 5.4 Create DIM_CATEGORY

Categories are embedded inside the product data.

Create:

```
models/marts/dimensions/dim_category.sql
```

```
SELECT DISTINCT

    category,

    ROW_NUMBER() OVER(
        ORDER BY category
    ) AS category_key

FROM {{ ref('stg_products') }}
```

Now every category has its own surrogate key.

Example:

category_key	category
1	Beauty
2	Furniture
3	Groceries

Step 5.5 Create DIM_COMPANY

The DummyJSON Users endpoint contains company information.

First, update stg_users.sql to include:

```
user_data:company.name::STRING AS company_name,
user_data:company.department::STRING AS department,
user_data:company.title::STRING AS job_title
```

Now create:

```
models/marts/dimensions/dim_company.sql
```

```
SELECT DISTINCT

    company_name,

    department,

    job_title,
```

```
    ROW_NUMBER() OVER(
        ORDER BY company_name
    ) AS company_key

FROM {{ ref('stg_users') }}
```

Step 5.6 Create DIM_ADDRESS

Update stg_users.sql with:

```
user_data:address.city::STRING AS city,
user_data:address.state::STRING AS state,
user_data:address.country::STRING AS country
```

Create:

```
models/marts/dimensions/dim_address.sql
```

```
SELECT DISTINCT

    city,

    state,

    country,

    ROW_NUMBER() OVER(
        ORDER BY city
    ) AS address_key

FROM {{ ref('stg_users') }}
```

Step 5.7 Create STG_CARTS

Before building the fact table, flatten the carts data.

Create:

```
models/staging/stg_carts.sql
```

```
WITH source AS (

    SELECT RAW_DATA
```

```
    FROM {{ source('raw','raw_carts') }}
),

flatten_carts AS (

    SELECT VALUE AS cart

    FROM source,
    LATERAL FLATTEN(input => RAW_DATA:carts)

)

SELECT

    cart:id::INTEGER AS cart_id,
    cart:userId::INTEGER AS user_id,
    cart:total::NUMBER AS total,
    cart:discountedTotal::NUMBER AS discounted_total,
    cart:totalProducts::INTEGER AS total_products,
    cart:totalQuantity::INTEGER AS total_quantity,
    cart:products AS products
FROM flatten_carts
```

Step 5.8 Create FACT_CARTS

Create:

```
models/marts/facts/fact_carts.sql
```

```
SELECT

    cart_id,
    user_id,
    total,
    discounted_total,
    total_products,
    total_quantity
FROM {{ ref('stg_carts') }}
```

This table stores one row per shopping cart.

Step 5.9 Create FACT_PRODUCT_SALES

Each cart contains multiple products. We need one row per product in a cart.

Create:

```
models/intermediate/int_cart_products.sql
```

```
WITH carts AS (

    SELECT *
    FROM {{ ref('stg_carts') }}
),

flatten_products AS (

    SELECT

        cart_id,

        user_id,
        VALUE AS product
    FROM carts,
    LATERAL FLATTEN(input => products)
)

SELECT

    cart_id,
    user_id,
    product:id::INTEGER AS product_id,
    product:quantity::INTEGER AS quantity,
    product:price::NUMBER AS price,
    product:discountedTotal::NUMBER AS discounted_total
FROM flatten_products
```

Now create:

```
models/marts/facts/fact_product_sales.sql
```

```
SELECT

    cart_id,
    user_id,
```

```
    product_id,
    quantity,
    price,
    discounted_total,
    quantity * price AS gross_sales
FROM {{ ref('int_cart_products') }}
```

This is the primary fact table for sales analysis.

Step 5.10 Run All Models

```
dbt run --profiles-dir .
```

Expected output:

```
Created model DIM_USERS

Created model DIM_PRODUCTS

Created model DIM_CATEGORY

Created model DIM_COMPANY

Created model DIM_ADDRESS

Created model STG_CARTS

Created model INT_CART_PRODUCTS

Created model FACT_CARTS

Created model FACT_PRODUCT_SALES

Completed successfully
```

Step 5.11 Verify the Relationships

You can now join your tables like this:

```
SELECT

    u.first_name,
    u.last_name,
    p.product_name,
```

```
    fps.quantity,
    fps.gross_sales

FROM FACT_PRODUCT_SALES fps

JOIN DIM_USERS u
ON fps.user_id = u.user_id

JOIN DIM_PRODUCTS p
ON fps.product_id = p.product_id;
```

This query combines customer, product, and sales data into a single analytical result.

📊 Final Star Schema

TABLE	TYPE	PURPOSE
DIM_USERS	Dimension	Customer information
DIM_PRODUCTS	Dimension	Product details
DIM_CATEGORY	Dimension	Product categories
DIM_COMPANY	Dimension	User company information
DIM_ADDRESS	Dimension	User location information
FACT_CARTS	Fact	Shopping cart summary
FACT_PRODUCT_SALES	Fact	Product-level sales transactions

📚 What You Learned

By completing Step 5, you have:

- Designed a dimensional model using the Star Schema pattern.
- Built reusable dimension tables from staging models.
- Created fact tables that store measurable business events.
- Flattened nested product arrays into individual sales records.
- Prepared a warehouse model optimized for Power BI and analytical queries.

🚀 STEP 6 — ORCHESTRATE THE PIPELINE WITH APACHE AIRFLOW

🎯 Objective

Instead of manually running:

```
python extract/extract_api.py

python snowflake/load_raw_data.py

dbt run --profiles-dir .

dbt test --profiles-dir .
```

Airflow will execute them automatically in the correct order.

Step 6.1 Project Structure

Your project should now include:

```
Capstone_Project/
│
├── airflow/
│   ├── dags/
│   │   └── ecommerce_pipeline.py
│   │
│   ├── logs/
│   ├── plugins/
│   └── docker-compose.yml
│
├── extract/
├── snowflake/
├── ecommerce_dbt/
├── data/
```

Step 6.2 Start Airflow

If you're using Docker:

```
docker compose up airflow-init
```

Then:

```
docker compose up -d
```

Verify the containers:

```
docker ps
```

You should see:

- airflow-webserver
- airflow-scheduler
- postgres
- redis

Step 6.3 Open Airflow

Open:

```
http://localhost:8080
```

Default credentials:

```
Username: airflow

Password: airflow
```

Step 6.4 Create Your First DAG

Create:

```
airflow/dags/ecommerce_pipeline.py
```

Step 6.5 Import the Required Libraries

```
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta
```

Step 6.6 Configure the DAG

```
default_args = {
    "owner": "Data Engineering",
    "depends_on_past": False,
    "retries": 2,
    "retry_delay": timedelta(minutes=2)
}
with DAG(
    dag_id="global_ecommerce_pipeline",
    default_args=default_args,
    start_date=datetime(2026, 1, 1),
    schedule="@daily",
    catchup=False,
    tags=["capstone", "snowflake", "dbt"]
) as dag:
```

Explanation

- **owner** → DAG owner.
- **retries** → Retry failed tasks twice.
- **retry_delay** → Wait 2 minutes before retrying.
- **schedule="@daily"** → Run once every day.
- **catchup=False** → Do not execute historical runs.

Step 6.7 Create the Extract Task

```
extract_api = BashOperator(
    task_id="extract_api",
    bash_command="""
```

```
    cd /opt/airflow/project &&
    python extract/extract_api.py
    """
)
```

This task downloads all datasets from the DummyJSON API.

Step 6.8 Create the Snowflake Load Task

```
load_snowflake = BashOperator(

    task_id="load_snowflake",

    bash_command="""
    cd /opt/airflow/project &&
    python snowflake/load_raw_data.py
    """
)
```

This loads the latest raw JSON files into the RAW schema.

Step 6.9 Create the dbt Run Task

```
dbt_run = BashOperator(

    task_id="dbt_run",

    bash_command="""
    cd /opt/airflow/project/ecommerce_dbt &&
    dbt run --profiles-dir .
    """
)
```

This builds all staging, intermediate, dimension, and fact models.

Step 6.10 Create the dbt Test Task

```
dbt_test = BashOperator(

    task_id="dbt_test",

    bash_command="""
```

```
    cd /opt/airflow/project/ecommerce_dbt &&
    dbt test --profiles-dir .
    """
)
```

This validates:

- Unique IDs
- Null checks
- Referential integrity
- Any additional tests you define

Step 6.11 Create a Data Quality Check

Add one more task:

```
quality_check = BashOperator(

    task_id="quality_check",

    bash_command="""
    echo 'Checking warehouse quality...'
    """
)
```

In a production pipeline, this task could run SQL queries to verify:

- Row counts
- Duplicate records
- Missing keys
- Freshness of the data

Step 6.12 Define the Task Dependencies

```
extract_api \
>> load_snowflake \
>> dbt_run \
>> dbt_test \
>> quality_check
```

The pipeline now executes in this order:

```
Extract API
      │
      ▼
Save Raw Files
      │
      ▼
Load Snowflake
      │
      ▼
Run dbt
      │
      ▼
Run dbt Tests
      │
      ▼
Quality Check
```

Step 6.13 View the DAG

Restart Airflow if necessary:

```
docker compose restart
```

Refresh the Airflow UI. You should see:

```
global_ecommerce_pipeline
```

Turn the DAG **ON** using the toggle switch.

Step 6.14 Trigger the Pipeline

Click:

```
▶ Trigger DAG
```

Airflow will execute each task in sequence.

Expected task statuses:

```
extract_api          ✅

load_snowflake       ✅

dbt_run              ✅

dbt_test             ✅

quality_check        ✅
```

Step 6.15 Monitor Logs

Click any task (e.g., dbt_run) and select **Logs**.

Example output:

```
Running model STG_USERS

Running model STG_PRODUCTS

Running model DIM_USERS

Running model FACT_PRODUCT_SALES

Completed successfully
```

Logs help diagnose issues if a task fails.

Step 6.16 Schedule the Pipeline

Airflow supports many schedules. Examples:

SCHEDULE	DESCRIPTION
@DAILY	Every day
@HOURLY	Every hour
@WEEKLY	Every week
"0 2 * * *"	Every day at 2:00 AM
"0 0 * * MON"	Every Monday at midnight

📊 Final Airflow Workflow

```
                DummyJSON API
                      |
                      ▼
          Extract Raw JSON Files
                      |
```

```
                    ▼
               Store in data/raw/
                    |
                    ▼
            Load into Snowflake RAW
                    |
                    ▼
                Run dbt Models
                    |
                    ▼
               Build Star Schema
                    |
                    ▼
               Run dbt Data Tests
                    |
                    ▼
              Perform Quality Checks
                    |
                    ▼
          Data Ready for Power BI
```

We can configure Airflow so that **airflow-init runs automatically as a one-time initialization service**. Then you only need to run:

```
docker compose up -d
```

The airflow-webserver and airflow-scheduler services will wait until initialization completes before starting. This is a cleaner setup for your capstone.

1. airflow/dags/ecommerce_pipeline.py

```
from datetime import datetime, timedelta

from airflow import DAG
from airflow.operators.bash import BashOperator

default_args = {
    "owner": "Data Engineering",
    "depends_on_past": False,
    "email_on_failure": False,
    "email_on_retry": False,
    "retries": 2,
    "retry_delay": timedelta(minutes=2),
```

```
}

with DAG(
    dag_id="global_ecommerce_pipeline",
    description="Global E-Commerce Analytics Pipeline",
    default_args=default_args,
    start_date=datetime(2026, 1, 1),
    schedule="@daily",
    catchup=False,
    max_active_runs=1,
    tags=["snowflake", "dbt", "airflow", "capstone"],
) as dag:

    # ------------------------------------------------------
    # Extract API Data
    # ------------------------------------------------------

    extract_api = BashOperator(
        task_id="extract_api",
        bash_command="""
        cd /opt/airflow/project &&
        python extract/extract_api.py
        """,
    )

    # ------------------------------------------------------
    # Load JSON into Snowflake
    # ------------------------------------------------------

    load_snowflake = BashOperator(
        task_id="load_snowflake",
        bash_command="""
        cd /opt/airflow/project &&
        python snowflake/load_raw_data.py
        """,
    )

    # ------------------------------------------------------
    # Run dbt Models
    # ------------------------------------------------------

    dbt_run = BashOperator(
        task_id="dbt_run",
        bash_command="""
        cd /opt/airflow/project/ecommerce_dbt &&
        dbt run --profiles-dir .
        """,
```

```
    )

    # ----------------------------------------------------
    # Run dbt Tests
    # ----------------------------------------------------

    dbt_test = BashOperator(
        task_id="dbt_test",
        bash_command="""
        cd /opt/airflow/project/ecommerce_dbt &&
        dbt test --profiles-dir .
        """,
    )

    # ----------------------------------------------------
    # Generate dbt Documentation
    # ----------------------------------------------------

    dbt_docs = BashOperator(
        task_id="dbt_docs",
        bash_command="""
        cd /opt/airflow/project/ecommerce_dbt &&
        dbt docs generate --profiles-dir .
        """,
    )

    # ----------------------------------------------------
    # Data Quality Check
    # ----------------------------------------------------

    quality_check = BashOperator(
        task_id="quality_check",
        bash_command="""
        echo "========================================"
        echo "Checking Data Warehouse Quality..."
        echo "Pipeline completed successfully."
        echo "========================================"
        """,
    )

    # ----------------------------------------------------
    # Workflow
    # ----------------------------------------------------

    (
        extract_api
        >> load_snowflake
```

```
        >> dbt_run
        >> dbt_test
        >> dbt_docs
        >> quality_check
    )
```

2. docker-compose.yml

This version uses the **latest Airflow image**, **PostgreSQL on port 5000**, and lets you start everything with a **single command**:

```
docker compose up -d
```

```
version: "3.9"

x-airflow-common: &airflow-common

  image: apache/airflow: 2.10.5

  env_file:
    - .env

  environment:

    AIRFLOW__CORE__EXECUTOR: LocalExecutor

    AIRFLOW__CORE__LOAD_EXAMPLES: "False"

    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:air
flow@postgres/airflow

    AIRFLOW__CORE__FERNET_KEY: ""

    AIRFLOW__WEBSERVER__SECRET_KEY: airflow

  volumes:

    - ./airflow/dags:/opt/airflow/dags

    - ./airflow/logs:/opt/airflow/logs

    - ./airflow/plugins:/opt/airflow/plugins

    - ./:/opt/airflow/project
```

```
  depends_on:
    postgres:
      condition: service_healthy

services:
  postgres:
    image: postgres:16
    container_name: airflow_postgres
    restart: always
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow
    ports:
      - "5000:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U airflow"]
      interval: 5s
      timeout: 5s
      retries: 10
    volumes:
      - postgres_data:/var/lib/postgresql/data
  airflow-init:
    <<: *airflow-common
    container_name: airflow_init
```

```
  command: >
    bash -c "
    airflow db migrate &&
    airflow users create
    --username airflow
    --firstname Admin
    --lastname User
    --role Admin
    --email admin@example.com
    --password airflow
    "
  restart: "no"

airflow-webserver:

  <<: *airflow-common

  container_name: airflow_webserver

  command: webserver

  ports:

    - "8080:8080"

  depends_on:

    airflow-init:
      condition: service_completed_successfully

  restart: always

airflow-scheduler:

  <<: *airflow-common

  container_name: airflow_scheduler

  command: scheduler

  depends_on:

    airflow-init:
      condition: service_completed_successfully

  restart: always
```

```
volumes:

  postgres_data:
```

Project Structure

Your project should now look like this:

```
Capstone_Project/
│
├── airflow/
│   ├── dags/
│   │   └── ecommerce_pipeline.py
│   ├── logs/
│   └── plugins/
│
├── data/
│   └── raw/
│
├── extract/
│   └── extract_api.py
│
├── snowflake/
│   ├── load_raw_data.py
│   ├── config.py
│   ├── create_database.sql
│   └── create_raw_tables.sql
│
├── ecommerce_dbt/
│
├── .env
│
├── docker-compose.yml
│
└── requirements.txt
```

Start the Entire Platform

Now you only need one command:

```
docker compose up -d
```

Open Airflow:

```
http://localhost:8080
```

Username

```
airflow
```

Password

```
airflow
```

This setup automatically initializes the Airflow metadata database, creates the admin user, starts PostgreSQL, the scheduler, and the webserver without requiring a separate airflow-init command.

📚 What You Learned

By completing Step 6, you have:

- Automated the end-to-end data pipeline with Apache Airflow.
- Built a DAG to orchestrate extraction, loading, transformation, and testing.
- Configured retries, scheduling, and logging.
- Created a production-style workflow that can run unattended.

🚀 STEP 7 — BUILD REPORTING TABLES

🎯 Objective

Create denormalized analytical tables that Power BI can query efficiently.

Instead of connecting Power BI directly to the fact and dimension tables, we'll create a dedicated **REPORTING** layer.

Step 7.1 Create a REPORTING Schema

In Snowflake, execute:

```
USE DATABASE CAPSTONE_DB;

CREATE SCHEMA IF NOT EXISTS REPORTING;
```

Your database architecture now becomes:

```
CAPSTONE_DB
│
├── RAW
├── STAGING
├── MART
└── REPORTING
```

Step 7.2 Create the Reporting Folder

Inside your dbt project:

```
models/
│
├── staging/
├── intermediate/
├── marts/
└── reporting/
```

All reporting models will be stored here.

Step 7.3 Configure dbt

Create:

```
models/reporting/schema.yml
```

```
version: 2

models:

  - name: rpt_daily_sales

  - name: rpt_monthly_sales

  - name: rpt_top_products
```

```
  - name: rpt_top_categories

  - name: rpt_customer_spending

  - name: rpt_average_order_value

  - name: rpt_company_distribution

  - name: rpt_posts_per_user

  - name: rpt_recipe_difficulty
```

Step 7.4 Reporting Table 1 — Customer Spending

Create:

```
models/reporting/rpt_customer_spending.sql
```

```
SELECT

    u.user_id,
    u.first_name,
    u.last_name,
    COUNT(f.cart_id) AS total_orders,
    SUM(f.discounted_total) AS total_spent,
    AVG(f.discounted_total) AS average_order_value
FROM {{ ref('fact_carts') }} f

JOIN {{ ref('dim_users') }} u
ON f.user_id = u.user_id

GROUP BY

    u.user_id,
    u.first_name,
    u.last_name
```

Example output:

Customer	Orders	Total Spent
John	5	2,450
Emily	3	1,320

Step 7.5 Reporting Table 2 — Top Products

Create:

`models/reporting/rpt_top_products.sql`

```
SELECT

    p.product_name,

    SUM(f.quantity) AS quantity_sold,

    SUM(f.gross_sales) AS revenue

FROM {{ ref('fact_product_sales') }} f

JOIN {{ ref('dim_products') }} p

ON f.product_id = p.product_id

GROUP BY

    p.product_name

ORDER BY revenue DESC
```

Step 7.6 Reporting Table 3 — Top Categories

`models/reporting/rpt_top_categories.sql`

```
SELECT

    p.category,

    SUM(f.gross_sales) AS total_sales,

    SUM(f.quantity) AS quantity_sold
```

```
FROM {{ ref('fact_product_sales') }} f

JOIN {{ ref('dim_products') }} p

ON f.product_id = p.product_id

GROUP BY

    p.category

ORDER BY total_sales DESC
```

Step 7.7 Reporting Table 4 — Company Distribution

```
models/reporting/rpt_company_distribution.sql
```

```
SELECT

    company_name,

    COUNT(*) AS employees

FROM {{ ref('dim_company') }}

GROUP BY company_name

ORDER BY employees DESC
```

Step 7.8 Reporting Table 5 — Average Order Value

```
models/reporting/rpt_average_order_value.sql
```

```
SELECT

    ROUND(AVG(discounted_total),2) AS average_order_value

FROM {{ ref('fact_carts') }}
```

Example:

Average Order Value

683.42

Step 7.9 Reporting Table 6 — Daily Sales

The DummyJSON carts endpoint does not include a purchase date. For demonstration purposes, we'll use the current date.

```
models/reporting/rpt_daily_sales.sql
```

```
SELECT

    CURRENT_DATE() AS sales_date,

    SUM(gross_sales) AS total_sales,

    COUNT(cart_id) AS total_orders

FROM {{ ref('fact_product_sales') }}

GROUP BY CURRENT_DATE()
```

Real-world note: In a production system, this table would use an actual order date from the source data rather than CURRENT_DATE().

Step 7.10 Reporting Table 7 — Monthly Sales

```
models/reporting/rpt_monthly_sales.sql
SELECT

    DATE_TRUNC('MONTH', CURRENT_DATE()) AS sales_month,

    SUM(gross_sales) AS revenue

FROM {{ ref('fact_product_sales') }}

GROUP BY DATE_TRUNC('MONTH', CURRENT_DATE())
```

Step 7.11 Reporting Table 8 — Products per Category

```
models/reporting/rpt_products_per_category.sql
```

```
SELECT

    category,

    COUNT(*) AS total_products

FROM {{ ref('dim_products') }}

GROUP BY category

ORDER BY total_products DESC
```

Step 7.12 Reporting Table 9 — Posts per User

First, create a staging model (stg_posts) similar to stg_users that flattens the posts JSON.

Then create:

```
models/reporting/rpt_posts_per_user.sql
```

```
SELECT

    user_id,

    COUNT(*) AS total_posts

FROM {{ ref('stg_posts') }}

GROUP BY user_id

ORDER BY total_posts DESC
```

Step 7.13 Reporting Table 10 — Recipe Difficulty

Similarly, create a staging model (stg_recipes) to flatten the recipes JSON.

Then create:

```
models/reporting/rpt_recipe_difficulty.sql
```

```
SELECT

    difficulty,

    COUNT(*) AS total_recipes

FROM {{ ref('stg_recipes') }}

GROUP BY difficulty

ORDER BY total_recipes DESC
```

Step 7.14 Configure REPORTING Materialization

Update dbt_project.yml:

```
models:

  ecommerce_dbt:

    reporting:

      +schema: REPORTING

      +materialized: table
```

This ensures all reporting models are created as physical tables in the REPORTING schema.

Step 7.15 Run dbt

Build only the reporting models:

```
dbt run --select reporting --profiles-dir .
```

Or rebuild the entire project:

```
dbt run --profiles-dir .
```

Step 7.16 Verify the Tables

In Snowflake:

```
USE DATABASE CAPSTONE_DB;
USE SCHEMA REPORTING;

SHOW TABLES;
```

You should see:

```
RPT_AVERAGE_ORDER_VALUE

RPT_COMPANY_DISTRIBUTION

RPT_CUSTOMER_SPENDING

RPT_DAILY_SALES

RPT_MONTHLY_SALES

RPT_POSTS_PER_USER

RPT_PRODUCTS_PER_CATEGORY

RPT_RECIPE_DIFFICULTY

RPT_TOP_CATEGORIES

RPT_TOP_PRODUCTS
```

In a production dbt project, **every raw table should have a corresponding staging model** before you build marts and reporting models.

This is the architecture we should have:

```
RAW
│
├── RAW_USERS
├── RAW_PRODUCTS
├── RAW_CARTS
├── RAW_POSTS
├── RAW_COMMENTS
├── RAW_RECIPES
├── RAW_QUOTES
```

```
└── RAW_CATEGORIES
        │
        ▼
STAGING
│
├── stg_users
├── stg_products
├── stg_carts
├── stg_posts
├── stg_comments
├── stg_recipes
├── stg_quotes
└── stg_categories
        │
        ▼
INTERMEDIATE
│
└── int_cart_products
        │
        ▼
MART
│
├── dim_users
├── dim_products
├── dim_category
├── dim_company
├── dim_address
├── fact_carts
└── fact_product_sales
        │
        ▼
REPORTING
```

Let's create all the missing staging models.

1. stg_posts.sql

models/staging/stg_posts.sql

```
WITH source AS (

    SELECT RAW_DATA
    FROM {{ source('raw', 'raw_posts') }}

),
```

```
flatten_posts AS (

    SELECT VALUE AS post_data
    FROM source,
    LATERAL FLATTEN(input => RAW_DATA:posts)

)

SELECT

    post_data:id::INTEGER           AS post_id,
    post_data:userId::INTEGER       AS user_id,
    post_data:title::STRING         AS title,
    post_data:body::STRING          AS body,
    post_data:views::INTEGER        AS views

FROM flatten_posts
```

2. stg_comments.sql

models/staging/stg_comments.sql

```
WITH source AS (

    SELECT RAW_DATA
    FROM {{ source('raw', 'raw_comments') }}

),

flatten_comments AS (

    SELECT VALUE AS comment_data
    FROM source,
    LATERAL FLATTEN(input => RAW_DATA:comments)

)

SELECT

    comment_data:id::INTEGER            AS comment_id,
    comment_data:body::STRING           AS comment,
    comment_data:postId::INTEGER        AS post_id,
    comment_data:user.id::INTEGER       AS user_id,
    comment_data:user.username::STRING  AS username

FROM flatten_comments
```

3. stg_quotes.sql

models/staging/stg_quotes.sql

```
WITH source AS (

    SELECT RAW_DATA
    FROM {{ source('raw', 'raw_quotes') }}

),

flatten_quotes AS (

    SELECT VALUE AS quote_data
    FROM source,
    LATERAL FLATTEN(input => RAW_DATA:quotes)

)

SELECT

    quote_data:id::INTEGER         AS quote_id,
    quote_data:quote::STRING       AS quote,
    quote_data:author::STRING      AS author

FROM flatten_quotes
```

4. stg_recipes.sql

models/staging/stg_recipes.sql

```
WITH source AS (

    SELECT RAW_DATA
    FROM {{ source('raw', 'raw_recipes') }}

),

flatten_recipes AS (

    SELECT VALUE AS recipe
    FROM source,
    LATERAL FLATTEN(input => RAW_DATA:recipes)
```

```
)

SELECT

    recipe:id::INTEGER                      AS recipe_id,
    recipe:name::STRING                     AS recipe_name,
    recipe:cuisine::STRING                  AS cuisine,
    recipe:difficulty::STRING               AS difficulty,
    recipe:prepTimeMinutes::INTEGER         AS prep_time_minutes,
    recipe:cookTimeMinutes::INTEGER         AS cook_time_minutes,
    recipe:servings::INTEGER                AS servings,
    recipe:caloriesPerServing::INTEGER      AS calories

FROM flatten_recipes
```

5. stg_categories.sql

The API endpoint /products/categories returns an array of categories. Since the structure can vary slightly, use this version:

```
WITH source AS (

    SELECT RAW_DATA
    FROM {{ source('raw', 'raw_categories') }}

),

flatten_categories AS (

    SELECT VALUE AS category
    FROM source,
    LATERAL FLATTEN(input => RAW_DATA)

)

SELECT

    category:name::STRING AS category_name,
    category:slug::STRING AS category_slug,
    category:url::STRING  AS category_url

FROM flatten_categories
```

6. Update schema.yml

Your models/staging/schema.yml should include all staging models.

```
version: 2

models:

  - name: stg_users

  - name: stg_products

  - name: stg_carts

  - name: stg_posts

  - name: stg_comments

  - name: stg_quotes

  - name: stg_recipes

  - name: stg_categories
```

7. Run dbt

First compile the staging models:

```
dbt run --select staging --profiles-dir .
```

Then build everything:

```
dbt run --profiles-dir .
```

Finally run all tests:

```
dbt test --profiles-dir .
```

📊 Final Reporting Architecture

```
DummyJSON API
       |
       ▼
```

```
RAW (JSON)
      |
      ▼
STAGING
      |
      ▼
DIMENSIONS
      |
      ▼
FACT TABLES
      |
      ▼
REPORTING TABLES
      |
      ▼
    Power BI
```

📚 What You Learned

By completing Step 7, you have:

- Created a dedicated reporting layer.
- Built business-friendly aggregation tables.
- Optimized data for dashboard performance.
- Separated analytical models from the underlying warehouse schema.
- Prepared the data model for Power BI.

🚀 STEP 8 — CI/CD WITH GITHUB ACTIONS

🎯 Objective

Automatically validate your project whenever code is pushed to GitHub.

Instead of manually running:

```
python extract/extract_api.py

python snowflake/load_raw_data.py
```

```
dbt run --profiles-dir .

dbt test --profiles-dir .
```

GitHub Actions will perform these checks automatically.

Final Project Structure

```
Capstone_Project/
│
├── .github/
│   └── workflows/
│       └── dbt_ci.yml
│
├── extract/
├── snowflake/
├── ecommerce_dbt/
├── airflow/
├── data/
├── requirements.txt
└── README.md
```

Step 9.1 Push Your Project to GitHub

Initialize Git (if you haven't already):

```
git init
```

Add all files:

```
git add .
```

Commit:

```
git commit -m "Initial Capstone Project"
```

Create a GitHub repository and push:

```
git remote add origin https://github.com/<username>/Capstone_Project.git
```

```
git branch -M main

git push -u origin main
```

Step 9.2 Create the Workflow Folder

```
.github/
└── workflows/
```

Inside it, create:

```
dbt_ci.yml
```

Step 9.3 Add GitHub Secrets

Open your GitHub repository.

Go to:

```
Settings
    ↓
Secrets and variables
    ↓
Actions
```

GitHub Repository → Settings → Secrets and variables → Actions

Create these repository secrets:

SECRET NAME	EXAMPLE
SNOWFLAKE_ACCOUNT	WQVQPDM-NV38719
SNOWFLAKE_USER	CRAFTSO...
SNOWFLAKE_PASSWORD	********
SNOWFLAKE_ROLE	ACCOUNTADMIN
SNOWFLAKE_WAREHOUSE	COMPUTE_WH

SNOWFLAKE_DATABASE	CAPSTONE_DB
MAIL_SERVER	smtp.gmail.com
MAIL_PORT	465
MAIL_USERNAME	your_email@gmail.com
MAIL_PASSWORD	Gmail App Password
MAIL_TO	your_email@gmail.com

After everything we've built, your project has grown into a **complete production-style data engineering project**.

```
Capstone_Project/
│
├── .github/
│   └── workflows/
│       └── dbt_ci.yml
│
├── airflow/
│   ├── dags/
│   │   └── ecommerce_pipeline.py
│   ├── logs/
│   ├── plugins/
│   ├── config/
│   └── docker-compose.yml
│
├── data/
│   ├── raw/
│   │   ├── users/
│   │   ├── products/
│   │   ├── carts/
│   │   ├── posts/
│   │   ├── comments/
│   │   ├── quotes/
│   │   ├── recipes/
│   │   └── categories/
│   │
│   └── processed/
│
├── extract/
│   ├── __init__.py
```

```
│   ├── extract_api.py
│   └── utils.py
│
├── snowflake/
│   ├── config.py
│   ├── create_database.sql
│   ├── create_raw_tables.sql
│   ├── load_raw_data.py
│   └── test_connection.py
│
├── ecommerce_dbt/
│   │
│   ├── analyses/
│   │
│   ├── macros/
│   │
│   ├── snapshots/
│   │
│   ├── seeds/
│   │
│   ├── tests/
│   │
│   ├── target/
│   │
│   ├── logs/
│   │
│   ├── dbt_packages/
│   │
│   ├── dbt_project.yml
│   │
│   └── models/
│       │
│       ├── sources/
│       │   └── sources.yml
│       │
│       ├── staging/
│       │   ├── schema.yml
│       │   ├── stg_users.sql
│       │   ├── stg_products.sql
│       │   ├── stg_carts.sql
│       │   ├── stg_posts.sql
│       │   ├── stg_comments.sql
│       │   ├── stg_quotes.sql
│       │   ├── stg_recipes.sql
│       │   └── stg_categories.sql
│       │
│       ├── intermediate/
```

```
│   │   └── int_cart_products.sql
│   │
│   ├── marts/
│   │   │
│   │   ├── dimensions/
│   │   │   ├── dim_users.sql
│   │   │   ├── dim_products.sql
│   │   │   ├── dim_category.sql
│   │   │   ├── dim_company.sql
│   │   │   └── dim_address.sql
│   │   │
│   │   └── facts/
│   │       ├── fact_carts.sql
│   │       └── fact_product_sales.sql
│   │
│   └── reporting/
│       ├── schema.yml
│       ├── rpt_average_order_value.sql
│       ├── rpt_company_distribution.sql
│       ├── rpt_customer_spending.sql
│       ├── rpt_daily_sales.sql
│       ├── rpt_monthly_sales.sql
│       ├── rpt_posts_per_user.sql
│       ├── rpt_products_per_category.sql
│       ├── rpt_recipe_difficulty.sql
│       ├── rpt_top_categories.sql
│       └── rpt_top_products.sql
│
├── powerbi/
│   ├── Global_Ecommerce_Analytics.pbix
│   ├── dashboard_design.pdf
│   └── screenshots/
│       ├── dashboard1.png
│       ├── dashboard2.png
│       └── dashboard3.png
│
├── docs/
│   ├── architecture.png
│   ├── star_schema.png
│   ├── airflow_dag.png
│   └── powerbi_dashboard.png
│
├── .env
├── .env.example
├── .flake8
├── .gitignore
├── .sqlfluff
```

```
├── pyproject.toml
├── requirements.txt
├── README.md
└── LICENSE
```

Step 9.4 Create the GitHub Actions Workflow

Create:

```
.github/workflows/dbt_ci.yml
```

```yaml
name: Capstone Data Engineering CI/CD

on:
  push:
    branches:
      - main

  pull_request:
    branches:
      - main

  workflow_dispatch:

env:
  SNOWFLAKE_ACCOUNT: ${{ secrets.SNOWFLAKE_ACCOUNT }}
  SNOWFLAKE_USER: ${{ secrets.SNOWFLAKE_USER }}
  SNOWFLAKE_PASSWORD: ${{ secrets.SNOWFLAKE_PASSWORD }}
  SNOWFLAKE_ROLE: ${{ secrets.SNOWFLAKE_ROLE }}
  SNOWFLAKE_WAREHOUSE: ${{ secrets.SNOWFLAKE_WAREHOUSE }}
  SNOWFLAKE_DATABASE: ${{ secrets.SNOWFLAKE_DATABASE }}

jobs:
  dbt-ci:

    runs-on: ubuntu-latest

    defaults:
      run:
        shell: bash

    steps:

      ########################################################
      # Checkout Repository
      ########################################################
```

```yaml
    - name: Checkout Repository
      uses: actions/checkout@v4

    ######################################################
    # Setup Python
    ######################################################

    - name: Setup Python
      uses: actions/setup-python@v5

      with:
        python-version: "3.12"

    ######################################################
    # Install Python Dependencies
    ######################################################

    - name: Install Dependencies
      run: |
        python -m pip install --upgrade pip
        pip install -r requirements.txt
    ######################################################
    # Install dbt
    ######################################################

    - name: Install dbt
      run: |
        pip install dbt-core dbt-snowflake
    ######################################################
    # Verify dbt
    ######################################################

    - name: Verify dbt Installation
      run: dbt --version

    ######################################################
    # Check Python Formatting
    ######################################################

    - name: Black Formatting Check
      run: |
        black --check .
    ######################################################
    # Python Linting
    ######################################################

    - name: Flake8 Lint
```

```
      run: |
        flake8 .
    ######################################################
    # SQL Linting
    ######################################################

    # - name: SQLFluff Lint
    #   run: |
    #     python -m sqlfluff lint ecommerce_dbt/models \
    #      --profiles-dir ~/.dbt

    ######################################################
    # dbt Debug
    ######################################################

    - name: dbt Debug
      run: |
        cd ecommerce_dbt
        dbt debug
    ######################################################
    # dbt Run
    ######################################################

    - name: dbt Run
      run: |
        cd ecommerce_dbt
        dbt run
    ######################################################
    # dbt Test
    ######################################################

    - name: dbt Test
      run: |
        cd ecommerce_dbt
        dbt test
    ######################################################
    # Generate Documentation
    ######################################################

    - name: Generate dbt Documentation
      run: |
        cd ecommerce_dbt
        dbt docs generate
    ######################################################
    # Upload dbt Documentation
    ######################################################
```

```
    - name: Upload dbt Artifacts
      uses: actions/upload-artifact@v4

      with:
        name: dbt-docs

        path: |
          ecommerce_dbt/target/index.html
          ecommerce_dbt/target/manifest.json
          ecommerce_dbt/target/catalog.json
          ecommerce_dbt/target/run_results.json
    ####################################################
    # Email Notification
    ####################################################

    - name: Send Email on Failure
      if: failure()

      uses: dawidd6/action-send-mail@v3

      with:
        server_address: ${{ secrets.MAIL_SERVER }}
        server_port: ${{ secrets.MAIL_PORT }}
        secure: true

        username: ${{ secrets.MAIL_USERNAME }}
        password: ${{ secrets.MAIL_PASSWORD }}

        subject: "❌ Capstone Pipeline Failed"

        to: ${{ secrets.MAIL_TO }}

        from: GitHub Actions

        body: |
          Your Capstone Data Engineering CI/CD Pipeline has failed.
          Repository:
          ${{ github.repository }}

          Branch:
          ${{ github.ref_name }}

          Workflow:
          ${{ github.workflow }}

          Commit:
```

```
            ${{ github.sha }}

            Actor:
            ${{ github.actor }}

            Please review the GitHub Actions logs.
```

Step 9.5 Update profiles.yml

Your local profiles.yml should reference environment variables:

```
ecommerce_dbt:

  target: dev

  outputs:

    dev:

      type: snowflake

      account: "{{ env_var('SNOWFLAKE_ACCOUNT') }}"
      user: "{{ env_var('SNOWFLAKE_USER') }}"
      password: "{{ env_var('SNOWFLAKE_PASSWORD') }}"

      role: "{{ env_var('SNOWFLAKE_ROLE') }}"
      warehouse: "{{ env_var('SNOWFLAKE_WAREHOUSE') }}"
      database: "{{ env_var('SNOWFLAKE_DATABASE') }}"

      schema: STAGING

      threads: 4
```

Step 9.6 Push Your Changes

```
git add .

git commit -m "Add GitHub Actions CI pipeline"

git push
```

GitHub Actions starts automatically.

Step 9.7 Monitor the Workflow

Open:

```
GitHub Repository

↓

Actions
```

You'll see:

```
dbt CI Pipeline
```

A successful run should look like:

```
✅ Checkout Repository

✅ Setup Python

✅ Install Dependencies

✅ Install dbt

✅ dbt run

✅ dbt test

SUCCESS
```

Step 9.8 Handle Failures

If a dbt model or test fails:

```
❌ dbt test failed

unique_stg_users_email

FAILED
```

The workflow stops immediately, preventing broken code from being merged.

We'll enhance the GitHub Actions workflow with:

- ✅ Python formatting check (black)
- ✅ Python linting (flake8)
- ✅ SQL linting (sqlfluff)
- ✅ Generate dbt documentation (dbt docs generate)
- ✅ Email notification on workflow failure

Step 9.10 Install Additional Dependencies

Update your requirements.txt to include:

```
requests
python-dotenv
snowflake-connector-python

dbt-core
dbt-snowflake

black
flake8
sqlfluff
sqlfluff-templater-dbt
```

Install locally:

```
pip install -r requirements.txt
```

Step 9.11 Configure Black

Create:

```
pyproject.toml
```

```
[tool.black]
line-length = 88
target-version = ["py312"]
```

Run locally:

```
black --check .
```

To automatically format your code:

```
black .
```

Step 9.12 Configure Flake8

Create:

```
.flake8
```

```
[flake8]
max-line-length = 88
extend-ignore = E203, W503
exclude =
    .git,
    .venv,
    target,
    dbt_packages,
    __pycache__
```

Run locally:

```
flake8 .
```

Step 9.13 Configure SQLFluff

Create:

```
.sqlfluff
```

```
[sqlfluff]
dialect = snowflake
templater = dbt

[sqlfluff:templater:dbt]
project_dir = ecommerce_dbt

[sqlfluff:rules]
max_line_length = 120
```

Run locally:

```
sqlfluff lint ecommerce_dbt/models
```

Automatically fix SQL formatting:

```
sqlfluff fix ecommerce_dbt/models
```

Step 9.14 Generate dbt Documentation

Add a new GitHub Actions step after dbt test:

```
- name: Generate dbt Documentation
  run: |
    cd ecommerce_dbt
    dbt docs generate --profiles-dir .
```

This generates:

```
target/

manifest.json

catalog.json

run_results.json

index.html
```

These files provide:

- Model lineage
- Column documentation
- Test results
- Dependencies
- Metadata

Step 9.15 Upload dbt Artifacts

Add:

```
- name: Upload dbt Artifacts
  uses: actions/upload-artifact@v4

  with:
    name: dbt-docs

    path: |
      ecommerce_dbt/target/index.html
      ecommerce_dbt/target/manifest.json
      ecommerce_dbt/target/catalog.json
      ecommerce_dbt/target/run_results.json
```

These artifacts are available for download after each successful workflow run.

Step 9.16 Add Black Check

Insert this step after installing dependencies:

```
- name: Check Python Formatting
  run: |
    black --check .
```

Step 9.17 Add Flake8 Linting

```
- name: Run Flake8
  run: |
    flake8 .
```

This catches issues such as:

- Unused imports
- Undefined variables
- Style violations
- Syntax errors

Step 9.18 Add SQLFluff

```
- name: SQLFluff Lint
  run: |
    sqlfluff lint ecommerce_dbt/models
```

This validates your dbt SQL models using the Snowflake dialect.

Step 9.19 Add Email Notification on Failure

If you're using a Gmail account with an App Password, add these repository secrets:

SECRET	DESCRIPTION
MAIL_SERVER	smtp.gmail.com
MAIL_PORT	465
MAIL_USERNAME	Your email address
MAIL_PASSWORD	Gmail App Password
MAIL_TO	Recipient email address

Then add the following step at the end of the workflow:

```
- name: Send Email on Failure
  if: failure()
  uses: dawidd6/action-send-mail@v3
  with:
    server_address: ${{ secrets.MAIL_SERVER }}
    server_port: ${{ secrets.MAIL_PORT }}
    secure: true
    username: ${{ secrets.MAIL_USERNAME }}
    password: ${{ secrets.MAIL_PASSWORD }}
    subject: "❌ Capstone CI Pipeline Failed"
    to: ${{ secrets.MAIL_TO }}
    from: GitHub Actions
    body: |
      The GitHub Actions workflow has failed.
      Repository: ${{ github.repository }}
      Branch: ${{ github.ref_name }}
      Commit: ${{ github.sha }}
```

```
      Workflow: ${{ github.workflow }}

      Check the workflow logs for details.
```

Security note: For Gmail, you should use an **App Password** rather than your normal account password.

Step 9.20 Final GitHub Actions Workflow

Your pipeline will now execute in this order:

```
Checkout Repository
        |
        ▼
Setup Python
        |
        ▼
Install Dependencies
        |
        ▼
Black Formatting Check
        |
        ▼
Flake8 Lint
        |
        ▼
SQLFluff Lint
        |
        ▼
Install dbt
        |
        ▼
dbt Run
        |
        ▼
dbt Test
        |
        ▼
dbt Docs Generate
        |
        ▼
Upload Documentation Artifacts
        |
        ▼
Success
```

```
                    │
                    └──────────────►
                                   On Failure
                                       │
                                       ▼
                                Email Notification
```

⭐ Final Enterprise CI/CD Features

FEATURE	STATUS
AUTOMATIC WORKFLOW TRIGGER ON PUSH/PR	✅
PYTHON DEPENDENCY INSTALLATION	✅
SNOWFLAKE ENVIRONMENT CONFIGURATION	✅
PYTHON FORMATTING (BLACK)	✅
PYTHON LINTING (FLAKE8)	✅
SQL LINTING (SQLFLUFF)	✅
DBT MODEL EXECUTION	✅
DBT TESTS	✅
DBT DOCUMENTATION GENERATION	✅
UPLOAD DBT DOCUMENTATION ARTIFACTS	✅
EMAIL NOTIFICATION ON FAILURE	✅

This enhanced workflow is much closer to what you'll see in enterprise data engineering teams and is an excellent addition to the project.

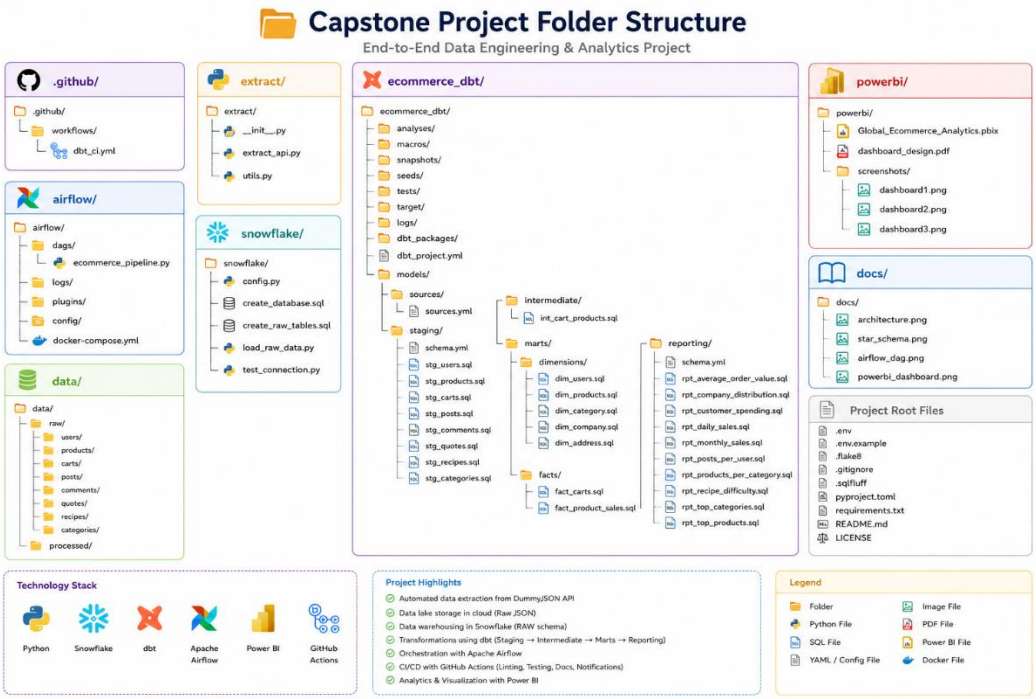
Final Folder Structure

```
Capstone_Project/
│
├── .github/
│   └── workflows/
│       └── dbt_ci.yml
│
```

```
├── airflow/
│   ├── dags/
│   │   └── ecommerce_pipeline.py
│   ├── logs/
│   ├── plugins/
│   ├── config/
│   └── docker-compose.yml
│
├── data/
│   ├── raw/
│   │   ├── users/
│   │   ├── products/
│   │   ├── carts/
│   │   ├── posts/
│   │   ├── comments/
│   │   ├── quotes/
│   │   ├── recipes/
│   │   └── categories/
│   │
│   └── processed/
│
├── extract/
│   ├── __init__.py
│   ├── extract_api.py
│   └── utils.py
│
├── snowflake/
│   ├── config.py
│   ├── create_database.sql
│   ├── create_raw_tables.sql
│   ├── load_raw_data.py
│   └── test_connection.py
│
├── ecommerce_dbt/
│   │
│   ├── analyses/
│   │
│   ├── macros/
│   │
│   ├── snapshots/
│   │
│   ├── seeds/
│   │
│   ├── tests/
│   │
│   ├── target/
│   │
```

```
    ├── logs/
    │
    ├── dbt_packages/
    │
    ├── dbt_project.yml
    │
    └── models/
        │
        ├── sources/
        │   └── sources.yml
        │
        ├── staging/
        │   ├── schema.yml
        │   ├── stg_users.sql
        │   ├── stg_products.sql
        │   ├── stg_carts.sql
        │   ├── stg_posts.sql
        │   ├── stg_comments.sql
        │   ├── stg_quotes.sql
        │   ├── stg_recipes.sql
        │   └── stg_categories.sql
        │
        ├── intermediate/
        │   └── int_cart_products.sql
        │
        ├── marts/
        │   │
        │   ├── dimensions/
        │   │   ├── dim_users.sql
        │   │   ├── dim_products.sql
        │   │   ├── dim_category.sql
        │   │   ├── dim_company.sql
        │   │   └── dim_address.sql
        │   │
        │   └── facts/
        │       ├── fact_carts.sql
        │       └── fact_product_sales.sql
        │
        └── reporting/
            ├── schema.yml
            ├── rpt_average_order_value.sql
            ├── rpt_company_distribution.sql
            ├── rpt_customer_spending.sql
            ├── rpt_daily_sales.sql
            ├── rpt_monthly_sales.sql
            ├── rpt_posts_per_user.sql
            ├── rpt_products_per_category.sql
            ├
```

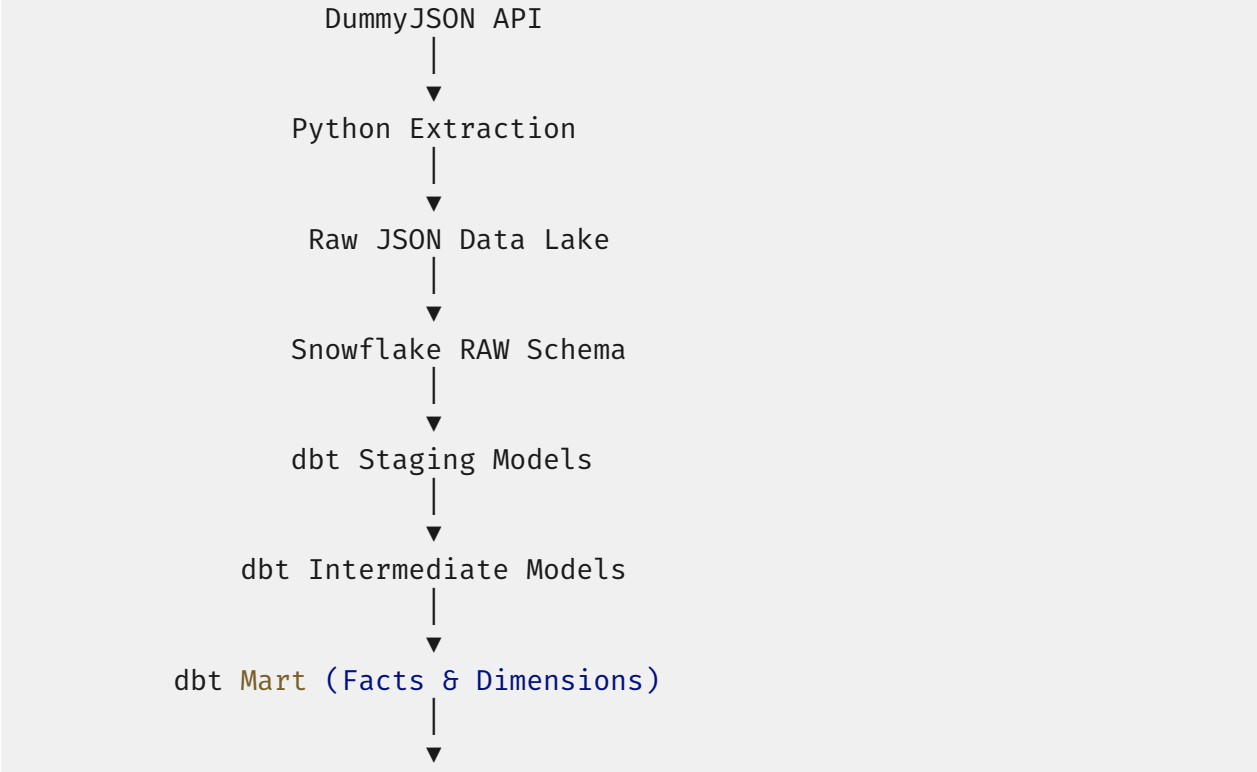
```
│           ├── rpt_recipe_difficulty.sql
│           ├── rpt_top_categories.sql
│           └── rpt_top_products.sql
│
├── powerbi/
│   ├── Global_Ecommerce_Analytics.pbix
│   ├── dashboard_design.pdf
│   └── screenshots/
│       ├── dashboard1.png
│       ├── dashboard2.png
│       └── dashboard3.png
│
├── docs/
│   ├── architecture.png
│   ├── star_schema.png
│   ├── airflow_dag.png
│   └── powerbi_dashboard.png
│
├── .env
├── .env.example
├── .flake8
├── .gitignore
├── .sqlfluff
├── pyproject.toml
├── requirements.txt
├── README.md
└── LICENSE
```



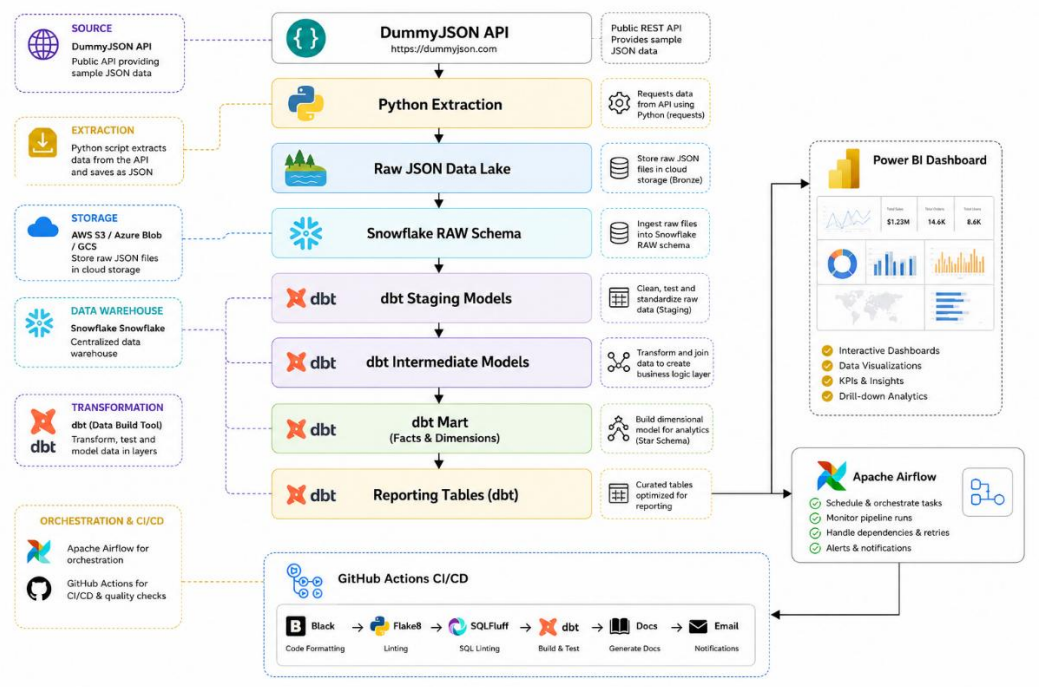
📁 Folder Purpose

FOLDER	PURPOSE
.GITHUB/WORKFLOWS	GitHub Actions CI/CD pipeline
AIRFLOW/	Airflow DAGs and Docker configuration
DATA/RAW	Raw API JSON files (Data Lake)
EXTRACT/	Python extraction scripts
SNOWFLAKE/	Snowflake SQL scripts and loaders
ECOMMERCE_DBT/	dbt project (transformations)
POWERBI/	Power BI report and dashboard assets
DOCS/	Architecture diagrams and screenshots

🏆 Final Architecture



```
              Reporting Tables (dbt)
                        |
          ┌─────────────┴─────────────┐
          ▼                           ▼
Apache Airflow                 Power BI Dashboard
          |
          ▼
GitHub Actions CI/CD
(Black • Flake8 • SQLFluff • dbt • Docs • Email)
```



⭐ What is the Project about?

For a portfolio project, **this structure is complete and professional**. It demonstrates:

- Modern Python data ingestion
- Cloud data warehousing with Snowflake
- Data transformation with dbt
- Workflow orchestration with Airflow
- Business intelligence with Power BI
- CI/CD with GitHub Actions

- Code quality (Black, Flake8, SQLFluff)
- Documentation and project organization

This is the kind of end-to-end project that closely resembles what many companies expect from a junior or mid-level data engineer.